INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR
Course IM60210 · Distributed Analytics for Industrial Automation

Distributed Control of a Robotic Assembly Line with Apache Hadoop

Mining plant-wide telemetry with MapReduce to close the control loop

Project Report

Submitted by	Sanika Kadam
Date of submission	2 November 2026
Source code	github.com/k-sanika-20/hadoop-line-control

Abstract

This project builds a distributed control system for a simulated six-station robotic assembly line in which Apache Hadoop is the plant-level supervisory tier (Level 3 of the ISA-95 automation hierarchy). A discrete-event simulator produces seven days of 1 Hz telemetry (4.84 million rows), a per-part event log and a maintenance log, which are stored in a three-DataNode HDFS cluster running in Docker. Four MapReduce jobs on YARN compute overall equipment effectiveness (J1, with a combiner over a custom Writable), the line constraint (J2, with a secondary sort), 3σ statistical process control limits (J3, with numerically stable parallel variance and mapper-side filtering) and tool-degradation curves (J4, with a map-side join). A synthesis step turns these aggregates into a new control policy (alarm limits, per-station preventive-maintenance intervals and buffer sizes) that is fed back to the line, closing the loop.

Validated against ground truth planted in the plant model, the jobs identify the true bottleneck (S03) and the fastest-wearing station (S04) and reproduce the simulator's per-station OEE exactly. The mined maintenance intervals range from 1,478 to 12,143 cycles, against a uniform hand-set 3,000, and land within 2–10% of the values implied by the true wear rates. In a paired evaluation over 20 random seeds on held-out days, the mined policy raises throughput by +1.01% (95% CI ±0.72%) and cuts maintenance downtime by 18.1% (±12.1%), at a cost of 0.47% in OEE. A fault-injection experiment shows a job finishing with byte-identical output after a worker node is killed part-way through.

Keywords: Apache Hadoop, MapReduce, HDFS, YARN, ISA-95, industrial automation, statistical process control, preventive maintenance, OEE, closed-loop control.

Contents

1 Introduction 2
2 Background and Related Work 2
3 System Architecture 3
4 Plant Simulator 4
5 Data Storage and the Hadoop Cluster 6
6 MapReduce Jobs 8
7 Policy Synthesis 10
8 Evaluation 11
9 Fault Tolerance 13
10 Dashboard 13
11 Discussion, Limitations and Future Work 14
12 Conclusion 14
References 15

1 Introduction

1.1 Motivation

A modern robotic assembly line reports a continuous stream of sensor readings from every robot: motor torque, temperature, vibration and load, many times a second. At the same time, the settings that govern the line, such as alarm limits on each sensor, how often each robot's tool is serviced and how much buffer space sits between stations, are usually chosen once by an engineer and are often the same for every station. Real stations are not the same: some wear their tools out several times faster than others, and one station is always the slowest, so the whole line waits on it.

The information needed to set each station's parameters correctly is present in the telemetry, but at plant scale the volume is far beyond what a person, a spreadsheet or a single machine can process. This is the problem that distributed analytics frameworks such as Apache Hadoop were built for [2–6].

1.2 Problem statement

Develop a distributed control system for a robotic manufacturing assembly line using Apache Hadoop. Concretely: store plant-wide telemetry in a distributed file system, mine it with MapReduce, convert the results into a new control policy for the line, apply that policy, and show with a fair statistical test whether the line performs better.

1.3 Reconciling batch analytics with control

A MapReduce job takes minutes; a robot's servo loop runs every few milliseconds. That looks like a contradiction until one notes that factory control is a *hierarchy* of loops on different timescales, formalised by the ISA-95 standard [1] (Table 1). Nobody would run a servo loop on Hadoop. Deciding, from weeks of telemetry across every robot, that one station's torque alarm should move or that its tool should be serviced twice as often is a large batch computation, and it is still control, because the number goes back down to the machine and changes what it does. Hadoop therefore sits at Level 3, where it sets the parameters that the fast Level 2 controllers enforce.

Table 1. *The ISA-95 automation hierarchy and where each part of the project sits.*

Level	Timescale	What is decided	In this project
1	1–10 ms	servo and PLC loops	inside each robot controller; out of scope
2	seconds	run / slow down / stop, per cell	six edge controllers in the simulator
3	hours–days	**what the setpoints and limits should be**	**Hadoop: four MapReduce jobs + policy synthesis**

2 Background and Related Work

2.1 Apache Hadoop

Hadoop is an open-source framework for storing and processing data sets too large for a single machine. It grew out of Google's published designs for the Google File System [3] and MapReduce [2]. Its central observation is that for large data the bottleneck is disk and network bandwidth rather than CPU, so it splits the data across many machines and ships the (small) program to where the data already is. It has three main parts [6, 17]:

- **HDFS** [4] stores files as fixed-size blocks (128 MB by default) spread across *DataNodes*, with each block replicated on several nodes. A single *NameNode* holds only metadata: the file tree, each file's blocks and each block's locations. DataNodes send heartbeats every three seconds; when a node falls silent, the NameNode re-replicates its blocks from the surviving copies.
- **YARN** [5] manages cluster resources. A *ResourceManager* schedules work onto *NodeManagers*, which run tasks inside memory- and CPU-bounded *containers*. Each job gets an *ApplicationMaster* that requests containers and re-runs failed tasks.

- **MapReduce** [2] expresses a computation as a *map* function applied to every input record independently, emitting key–value pairs, and a *reduce* function applied to all values sharing a key. Between them the framework *shuffles* and sorts by key across the network. An optional *combiner* pre-aggregates map output locally, and the scheduler prefers to run each map task on a node holding its input block (*data locality*).

2.2 Manufacturing concepts used

- **Overall equipment effectiveness (OEE)** [7] = Availability × Performance × Quality: the share of planned time the machine was up, how close to its ideal speed it ran, and the share of good parts.
- **The line constraint.** In a serial line with finite buffers, the slowest station limits the output of the whole line [8, 9]. Stations upstream of it become *blocked* (no room to pass a part on) and stations downstream become *starved* (no part to work on). The constraint is the station that is idle least.
- **Statistical process control (SPC)** [10]: a process is monitored against control limits at mean ± kσ (commonly k = 3); a reading outside them signals a probable assignable cause.
- **Preventive vs corrective maintenance.** Servicing a tool on a schedule costs a short planned stop; letting it run until it fails costs a long unplanned repair and more scrap on the way. The right interval depends on each tool's wear rate.

2.3 Related work

Hadoop and MapReduce have been applied to industrial sensor data in several ways. Bahga and Madisetti's CloudView [12] stores machine sensor data in the cloud and uses MapReduce jobs to build a case base of past faults, against which live readings are matched to predict incipient faults. Bao et al. [13] present a Hadoop-based framework for the parallel storage and processing of massive sensor data in cloud manufacturing. For quality and process monitoring, Zheng et al. [14] train a Bayesian network for manufacturing quality control with MapReduce, and Zhu et al. [15] fit a distributed Gaussian mixture model on Hadoop to monitor plant-wide processes with several operating modes. Patwardhan et al. [16] survey the shift from preventive to predictive maintenance enabled by big-data processing.

These works largely stop at *monitoring*: they detect a fault, predict a failure or explain quality variation. The present project differs in two ways. First, it **closes the loop**: the mined aggregates become the next control policy, the plant is re-run under it, and the effect is measured with a paired statistical test. Second, because the plant is simulated with **planted ground truth**, every analytical result can be checked against a known answer, which is not possible with unlabelled plant data.

3 System Architecture

Figure 1 shows the system against the ISA-95 levels. The six stations and their edge controllers form Level 2; the Hadoop cluster and the policy synthesis form Level 3. Data flows up as telemetry and event logs; decisions flow down as a policy file.

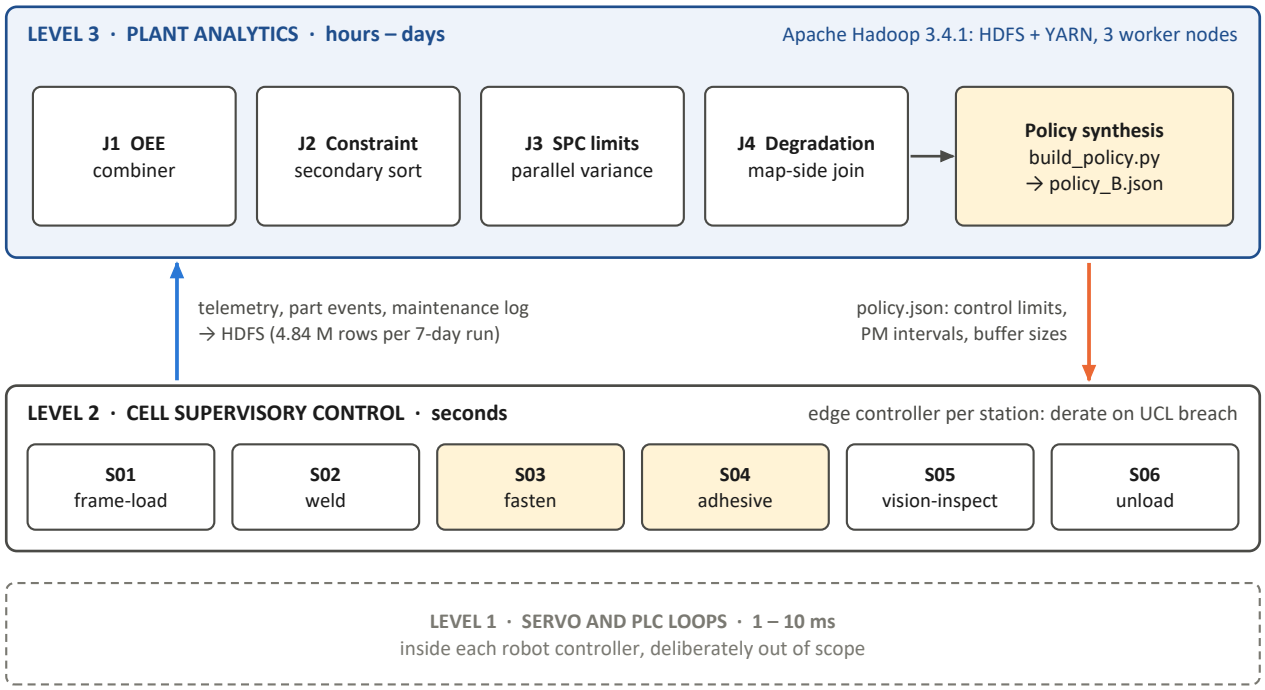


Figure 1. *System architecture. Hadoop is the Level 3 tier; it never drives a robot directly, but it sets the limits, maintenance intervals and buffer sizes that Level 2 enforces. S03 and S04 are highlighted as the planted bottleneck and fastest-wearing station.*

4 Plant Simulator

No public data set offers robot telemetry together with the ability to change the control policy and re-run the plant, which closing the loop requires. The line is therefore simulated. The simulator is a discrete-event model of a six-station serial line with finite buffers (sim/line_sim.py). It is fully seeded, so the same seed always reproduces the same run.

4.1 Line structure

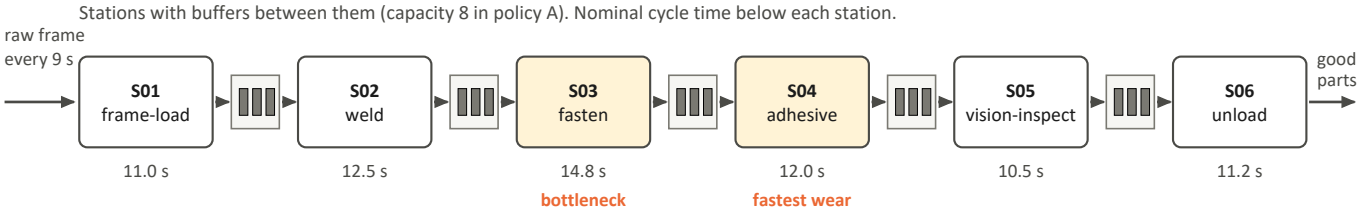


Figure 2. *The simulated line. A product passes through six stations in order; parts queue in a finite buffer between each pair.*

Table 2. *Physical parameters of the plant. These are not policy: the controller cannot change them, and the MapReduce jobs never see them.*

Station	Operation	Nominal cycle (s)	Torque (Nm)	Noise σ (Nm)	Wear rate (per cycle)	Base defect rate
S01	frame-load	11.0	28	1.10	0.55×10^{-4}	0.35%
S02	weld	12.5	44	2.30	1.10×10^{-4}	0.60%
S03	fasten	14.8	39	1.70	0.90×10^{-4}	0.50%
S04	adhesive	12.0	33	1.45	2.30×10^{-4}	0.55%
S05	vision-inspect	10.5	18	0.80	0.30×10^{-4}	0.15%
S06	unload	11.2	24	1.00	0.50×10^{-4}	0.25%

Two facts are planted in these parameters. **S03 has the longest nominal cycle**, so it is the true bottleneck; **S04 has the highest wear rate** (about 4× S01's), so it degrades fastest and needs the shortest maintenance interval. A correct bottleneck job must find S03 and a correct degradation job must find S04. The parameter values are synthetic: chosen to be physically plausible in size and to embed this ground truth, not taken from a particular plant.

4.2 Line dynamics

Each simulated day is one 8-hour shift (28,800 s). Raw frames arrive every 9 s, faster than any station, so the line is limited by its constraint rather than by supply. For part n at station i, processing starts at the latest of three times: when the part arrives from station i−1, when station i finished part n−1, and when there is room in the downstream buffer, i.e. when part n−b−1 has left station i+1, where b is the buffer capacity. Time spent waiting on the first condition is recorded as *starved*, on the third as *blocked*. This departure-time recursion is the standard model of a tandem line with blocking [9].

4.3 Tool wear

Each station's wear grows linearly with cycles since its last maintenance and is reset to zero by any maintenance action:

$$w = \min(1,\ c \times r \times 1.5)$$

where c is cycles since maintenance and r the station's wear rate. Wear acts on the process in four ways:

- **Torque** rises by up to 30%: $\tau = \tau_0 \times \text{speed} \times (1 + 0.30w) + \varepsilon$, with Gaussian measurement noise $\varepsilon \sim N(0, \sigma)$.
- **Cycle time** rises by up to 22%: $t = (t_{nom} / \text{speed}) \times (1 + 0.22w) \times L$, where L is a lognormal factor with about 6% spread (task times are positive and right-skewed).
- **Defect probability** rises: $p = p_0 + 0.075w + 0.045 \times \max(0, \text{speed} - 1)$, multiplied by 0.55 while the station is derated.
- **Failure hazard** per cycle rises steeply: $h = 2{\times}10^{-5} + 9{\times}10^{-4}\, w^3$. The cubic term mimics a Weibull wear-out hazard (shape 4); a fully worn tool is about 46× more likely to fail than a fresh one.

The other three sensors (motor temperature, vibration with heavy-tailed spikes, spindle load) also rise with wear and carry their own noise. While a station is stopped, all its sensors keep reporting, but torque drops to about 5% of normal; Section 6 shows why these readings must be filtered.

4.4 Maintenance and the edge controller

A **preventive** stop happens every N cycles (from the policy) and lasts 240 s. A **corrective** stop happens when the failure hazard fires and lasts an exponentially distributed time with mean 1,500 s, about six times longer. Both reset wear. After every cycle, the station's **edge controller** (Level 2) compares the expected torque with the policy's upper control limit; on a breach it derates the station to 85% speed for the next 40 cycles, which cuts

defects at the cost of output. The control limits therefore genuinely change how the line behaves.

4.5 The control policy

Everything the line obeys is read from a JSON policy file; nothing is hard-coded. Table 3 lists the fields. Policy A, the baseline, is what an engineer might set by hand: the same for every station.

Table 3. *Fields of the policy file, and the baseline policy A.*

Field	Effect in the simulator	Policy A (all stations)
speed_setpoint	divides cycle time; over-speed adds defects	1.0
torque_ucl / torque_lcl	a breach of the UCL derates the station for 40 cycles	nominal torque ± 4σ
pm_interval_cycles	preventive maintenance every N cycles	3,000
buffer_capacity	size of the buffer after the station	8
ideal_cycle_s	reference for OEE performance (not a control)	nominal cycle time

4.6 Outputs

Table 4. *Simulator outputs. Output is plain CSV because gzip files cannot be split across mappers.*

Output	Schema	Size (7 days, 1 Hz)	Consumed by
telemetry/dt=…/station=…/ part-0.csv	ts_ms, station, signal, value	4,838,400 rows, ~145 MB	J3, J4
part_events/dt=…/part-0.csv	ts_end_ms, station, part_id, cycle_ms, blocked_ms, starved_ms, down_ms, status, defect_code	74,460 rows, 2.9 MB	J1, J2
maintenance/maintenance.csv	ts_ms, station, event, duration_ms	25 rows	J4 (map-side join)
run_summary.json	simulator's own per-station totals	1 file	validation only; never given to Hadoop

5 Data Storage and the Hadoop Cluster

5.1 Cluster

The cluster runs as six Docker containers on one network, sized to fit an 8 GB Docker allocation on a laptop (Figure 3). A client container issues all commands; the NameNode and ResourceManager run in their own containers; each of three workers runs a DataNode and a NodeManager. The configuration (hadoop-conf/) is mounted into every container; the values that matter are in Table 5.

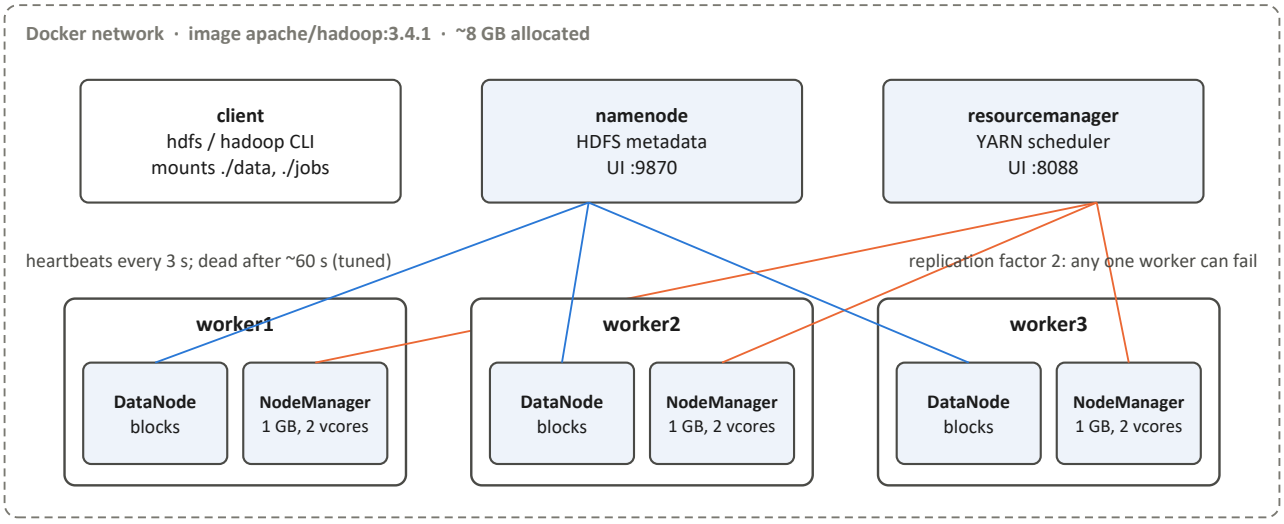


Figure 3. *Cluster topology: one master per service, three workers, replication factor 2.*

Table 5. *Key cluster configuration.*

Setting	Value	Why
dfs.replication	2	survives the loss of any one of three DataNodes
dfs.blocksize	128 MB (default)	each telemetry file is one block and one split
yarn.nodemanager.resource.memory-mb / cpu-vcores	1024 / 2	fits 8 GB for 6 containers
mapreduce.map.memory.mb / reduce	512 / 1024	2 map containers per worker
dfs.namenode.heartbeat.recheck-interval	15 s	DataNode declared dead after ~60 s instead of 10.5 min (demo)
yarn.nm.liveness-monitor.expiry-interval-ms	60 s	NodeManager declared lost after 60 s (demo)

5.2 Data volume and layout

One reading per second over an 8-hour shift gives 28,800 readings per sensor per day; × 4 sensors × 6 stations × 7 days = **4,838,400** telemetry rows, about 145 MB.

Telemetry is written in long format (one reading per row), so a mapper can keep one sensor with a single comparison, and new sensors need no schema change. Files are partitioned by day and station in Hive-style directories (dt=…/station=…), so a job can read a subset, e.g. one station or one day, by path alone. This gives 6 stations × 7 days = **42 files** of about 3.5 MB each. Every file fits in one 128 MB block, so each file is one input split and one map task: the parallelism comes from the 42 files. Together with 7 part-event files and the maintenance log, HDFS holds 50 files (154.5 MB) as 50 blocks and 100 replicas; fsck reports the file system as HEALTHY with no under-replicated blocks.

The files are deliberately left uncompressed: a gzip file cannot be split, so it would be read by a single mapper and the block-level parallelism would be lost. The opposite risk, the *small-files problem* (NameNode memory and task start-up costs growing with the number of files), does not arise at this scale; at plant scale the files would be grown towards the block size, or packed with CombineFileInputFormat.

6 MapReduce Jobs

The four jobs are packaged in one jar, line-control-jobs.jar (45 KB), whose entry point is a Hadoop ProgramDriver, so each job is called by name: `hadoop jar line-control-jobs.jar limits <in> <out>`. Every job is a ToolRunner driver, so options such as `-D lc.signal=torque` work from the command line, and every job counts the rows it keeps and skips. Each job was chosen to compute something the control system needs *and* to demonstrate a different MapReduce technique.

6.1 J1: Overall equipment effectiveness (combiner over a custom Writable)

J1 reads the part-event log. The mapper parses each row and emits the station as key and an OeeWritable as value, holding additive sufficient statistics: parts, good parts, summed cycle time, downtime, blocked and starved time and the minimum cycle. Because every field is additive, a **combiner** can fold partial results on each mapper and the result is bit-identical; 74,460 map records shrink to 42 before the shuffle. The reducer forms the ratios once, at the end; averaging per-cycle ratios would give a different and wrong number.

$$A = (T_{planned} - T_{down}) / T_{planned} \qquad P = (t_{ideal} \times parts) / \Sigma\, t_{cycle}$$

$$Q = good / parts \qquad OEE = A \times P \times Q$$

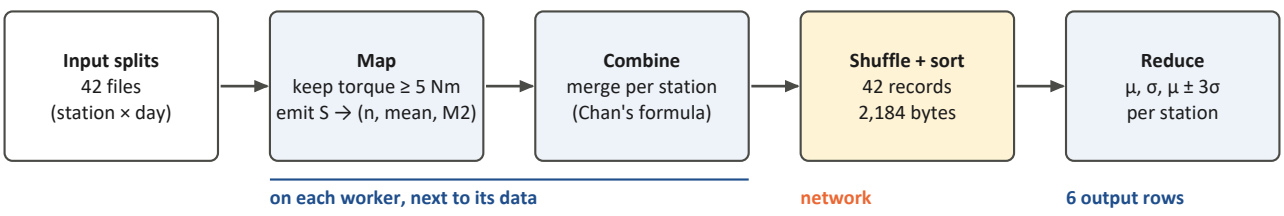
The ideal (nameplate) cycle time of each station is passed in by mine.ps1 as -D lc.ideal.S01=11.0 and so on, from the policy file; if it is missing the job falls back to the best demonstrated cycle and says so in a counter.

6.2 J2: The line constraint (secondary sort)

Counting rows per station cannot find a bottleneck: in a serial line every station processes almost the same number of parts. The constraint is the station that is idle least. J2 computes utilisation = run time / (run + blocked + starved time) and also the longest unbroken run of blocked cycles, which needs each station's records in time order. A station has millions of records, so buffering and sorting them in the reducer would exhaust its memory. Instead J2 uses a **secondary sort**: a composite key (station, timestamp), a partitioner and a grouping comparator on the station alone, and a sort comparator on the full key, so the shuffle delivers each station's records to one reduce call already in time order. J2 deliberately has **no combiner**, because folding records map-side would destroy that order; the job counter *Combine input records = 0* is the evidence.

6.3 J3: SPC control limits (parallel variance, mapper-side filtering)

J3 computes mean, σ and k-σ control limits of a chosen sensor per station (Figure 4). Two filters run in the mapper. First, telemetry interleaves four sensors on very different scales (torque about 40, temperature about 60, vibration about 0.5, load about 62), so only the requested signal is kept; this alone removes 75% of the data before it reaches the network. Second, readings taken while the machine was stopped are dropped (torque below 5 Nm, set with -D lc.min.value=5.0).



4.84 M rows in → 3.63 M dropped (other signals) and 8,587 dropped (machine stopped) in the mapper → 52.8 MB of map output → 2,184 bytes shuffled.

Figure 4. *Data flow of J3. Filtering and the combiner reduce 52.8 MB of map output to 2,184 bytes crossing the network.*

Each partial result is a triple (n, mean, M_2), where M_2 is the sum of squared deviations. Partials are merged with Chan's parallel formula [11]:

$$n = n_a + n_b \quad \delta = mean_b - mean_a \quad mean = mean_a + \delta\, n_b / n$$
$$M_2 = M_{2,a} + M_{2,b} + \delta^2\, n_a\, n_b / n \quad \sigma = \sqrt{(M_2 / n)}$$

The textbook shortcut $(\Sigma x^2 - (\Sigma x)^2/n)/n$ subtracts two large, nearly equal numbers and can even return a negative variance for data centred far from zero, such as motor temperature. Chan's update is numerically stable and associative, so it works as both combiner and reducer.

6.4 J4: Degradation curves (map-side join)

A degradation curve plots torque against *time since the last maintenance*, which requires joining every telemetry row with the most recent maintenance event before it. The maintenance log has 25 rows; telemetry has millions. J4 therefore ships the small table to every mapper through the **distributed cache** and performs the join in memory (a **map-side join**): in setup() each mapper loads the maintenance times per station, and for each torque reading a binary search finds the last maintenance at or before it. The reading's age is bucketed into 30-minute bins and emitted with key (station, bucket). A reduce-side join would instead push the entire telemetry table across the network to achieve the same thing. Readings before a station's first maintenance are skipped and counted, because their age is unknown rather than zero.

6.5 Measured job counters

Table 6. *Counters reported by the jobs.*

Measurement (7-day run)	Value
J3 combiner: map output → bytes shuffled	52.8 MB → 2,184 bytes (1.20 M records → 42)
J3 rows dropped in the mapper as other signals	75.1%
J3/J4 rows dropped as machine stopped	8,587 (vs 8,604 s of simulated downtime)
J3 malformed rows	0
J1 combiner	74,460 records → 42
J2 combiner input records	0 (deliberately no combiner)
Data-local map tasks (J4)	42 of 42 on the latest run (33–42 across runs)

The stopped-machine filter is cross-checked against the simulator: the 8,587 torque samples removed agree within 0.2% with the 2.39 h (8,604 s) of downtime the simulator recorded, so the filter removes the stopped time and nothing else.

Table 7. *J3 output: torque statistics and 3σ control limits per station. S04 has fewer running samples because it was down longest.*

Station	n	Mean (Nm)	σ (Nm)	LCL	UCL
S01	200,640	29.02	1.262	25.24	32.81
S02	200,640	47.23	3.005	38.21	56.24
S03	200,640	41.33	2.200	34.73	47.93
S04	197,813	37.33	3.257	27.56	47.10
S05	200,640	18.36	0.828	15.88	20.84
S06	200,640	24.80	1.110	21.47	28.13

7 Policy Synthesis

sim/build_policy.py reads the four job outputs, about two dozen rows, and writes the next policy. This step is deliberately **not** a MapReduce job: pushing kilobytes through YARN to produce kilobytes would add overhead and no value. The batch tier is where the aggregation happens; synthesis is a control decision taken on the aggregate. Each job drives one part of the policy:

- **J3 → control limits:** torque_lcl / torque_ucl = mean ∓ 3σ.
- **J4 → maintenance intervals.** A straight line is fitted by least squares to mean torque against age, using buckets with at least 20% of the fullest bucket's samples (the thin tail is noise). Its intercept is the fresh torque τ_0 and its slope s the wear rate in Nm per hour. A tool is serviced once torque has risen 15% above fresh, the one tuning parameter of the synthesis.
- **J2 → buffers:** the buffers on either side of the constraint (after the station feeding it and after the constraint itself) are enlarged by 50%, so the constraint is neither starved nor blocked.
- **J1** is reported for context and drives nothing on its own.

The interval follows from the slope, converted from hours to cycles through each station's wall-clock time per cycle:

$$PM_{hours} = 0.15\,\tau_0 / s \qquad PM_{cycles} = PM_{hours} \times 3600 \times utilisation / t_{cycle} \qquad \textit{(clamped to 400 – 20,000)}$$

The factor utilisation / t_{cycle} matters: the degradation curve's x-axis is wall-clock age, but the interval is counted in cycles, and a station spends 4–34% of the clock blocked or starved. Ignoring this overstates the interval by 1/utilisation (Section 8.5).

Table 8. *Policy A (hand-set) vs policy B (mined). Speed setpoints are unchanged (1.0).*

Station	UCL A	UCL B	PM A	PM B	Buffer A	Buffer B
S01	32.4	32.8	3,000	**6,177**	8	8
S02	53.2	56.2	3,000	**3,139**	8	**12**
S03	45.8	47.9	3,000	**3,888**	8	**12**
S04	38.8	47.1	3,000	**1,478**	8	8
S05	21.2	20.8	3,000	**12,143**	8	8
S06	28.0	28.1	3,000	**7,353**	8	8

Table 9. *Mined maintenance intervals against the intervals implied by the simulator's true wear rates (torque +15%, i.e. wear 0.5). Hadoop never sees the true rates.*

Station	J4 slope (Nm/h)	Mined PM (cycles)	True PM (cycles)	Error
S01	0.152	6,177	6,061	+1.9%
S02	0.471	3,139	3,030	+3.6%
S03	0.337	3,888	3,704	+5.0%
S04	0.763	1,478	1,449	+2.0%
S05	0.050	12,143	11,111	+9.3%
S06	0.109	7,353	6,667	+10.3%

The mined intervals land within 2–10% of the truth, and for S04, the station that matters most, within 2%. The small, consistent overestimate comes from the residual flattening of the fitted slope by noise in the high-age buckets.

8 Evaluation

8.1 Validation against planted ground truth

Table 10. *Every analytical result checked against a known answer.*

Check	Planted / true	Mined	Result
Line constraint (J2)	S03	S03	utilisation 0.957, highest on the line
Fastest degradation (J4)	S04	S04	slope 0.763 Nm/h, 15× the slowest (S05)
Per-station OEE (J1)	run_summary.json	J1 output	exact match, all six stations
Maintenance intervals	true wear rates	policy B	within 2–10% (Table 9)

8.2 What the analytics show

Figure 5 shows the fingerprint of a constraint. Stations upstream of S03 are mostly *blocked*, waiting for it to make room; stations downstream are mostly *starved*, waiting for it to send parts; S03 itself is busy 96% of the time. This is exactly how J2 identifies it without being told any cycle times.

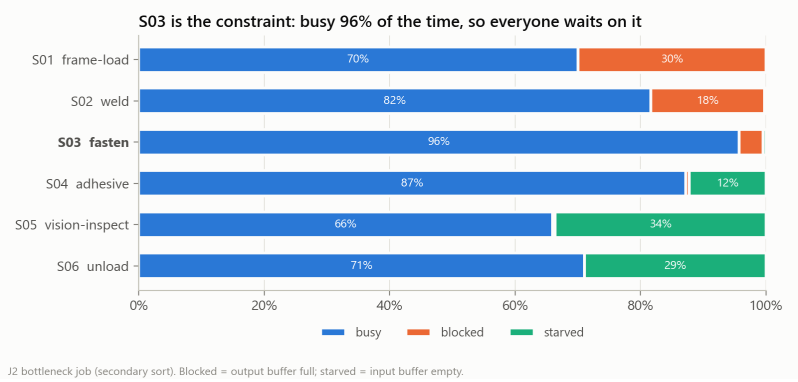


Figure 5. *Share of each station's time spent busy, blocked and starved (J2).*

J1's OEE breakdown points at the same station from another angle. S04 is lowest on all three factors: it breaks down (availability), runs slow while worn (performance) and makes more defects (quality), because under policy A it reaches full wear at about 2,900 cycles, before its 3,000-cycle service.

Figure 6 plots J4's degradation curves. Torque climbs steadily with time since maintenance, fastest for S04. The slopes are what set the maintenance intervals in Figure 7: instead of one number for every station, the mined policy spreads the intervals over an 8× range, servicing S04 twice as often and S05 four times less often.

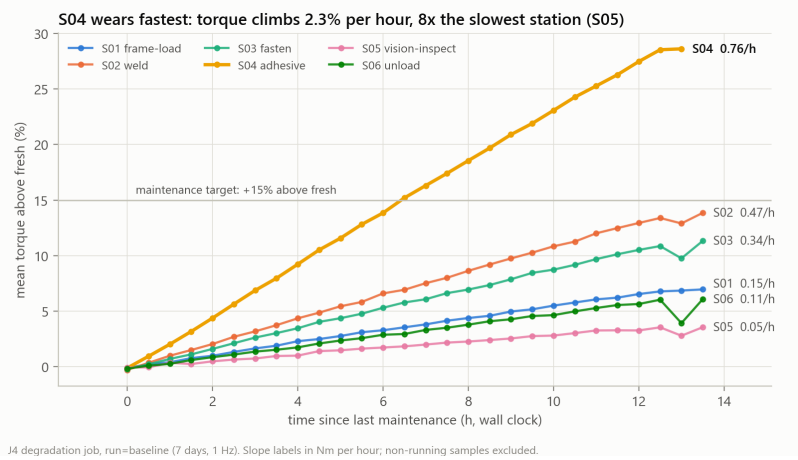


Figure 6. *Mean torque above fresh against time since maintenance (J4), buckets used in the slope fit.*

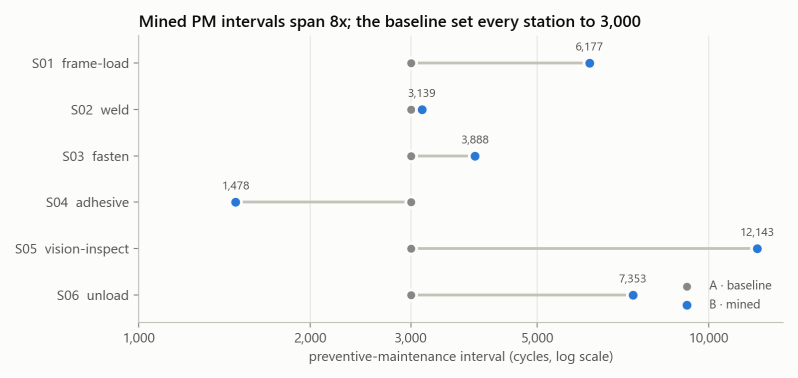


Figure 7. *Preventive-maintenance intervals: uniform baseline A against mined policy B (log scale).*

8.3 Closed-loop experiment design

Policy B was mined from the 7-day baseline run (seed 42). To evaluate it, run_ab.py simulates 30 days under policy A and under policy B for each of 20 seeds (1–20), none of which was used for mining. Within a seed, both policies face the same random stream, so differences are caused by the policy rather than by luck (*paired* design). Only days 21–30 are scored, so each line is compared after it has settled into its policy's maintenance rhythm. For each metric the mean paired difference is reported with a 95% confidence interval, $1.96 \times s_d / \sqrt{20}$; a difference is significant when the interval excludes zero.

Table 11. *Closed-loop results, 20 paired seeds, days 21–30 of 30.*

Metric	A · baseline	B · mined	Δ (B – A)	95% CI	Change	Verdict
throughput (good parts)	17,259.5	17,433.5	+174.1	±124.7	**+1.01%**	B better
maintenance downtime (h)	4.776	3.913	-0.864	±0.579	**-18.08%**	B better
line OEE	0.9219	0.9175	-0.0043	±0.0013	**-0.47%**	B worse

8.4 Interpretation

Policy B produces +1.01% more good parts and 18.1% less maintenance downtime (planned services and breakdowns together). The downtime gain is the larger effect: S04 no longer runs to failure, and slow-wearing stations such as S05 stop being serviced long before they need it. Line OEE, however, falls slightly (-0.47%). This is a genuine trade-off rather than a defect: stretching the interval on slow-wearing stations saves stop time but lets more wear accumulate between services, which lengthens cycle times and costs performance. The synthesis optimises one stated target (service at 15% torque rise); the dashboard's what-if controls let this target be moved and re-scored. A result in which every metric improved would deserve more suspicion than this mixed one.

8.5 The first mined policy was worse than the baseline

The first version of the pipeline produced a policy that scored −3.8% throughput and +21.6% maintenance downtime against the baseline. The ground-truth checks traced this to two independent measurement errors that multiplied:

1. **Stopped machines still emit telemetry.** Near-zero torque readings from stopped stations landed in the highest age buckets (stops happen at the end of a wear cycle) and flattened the fitted slopes. They also inflated σ by 1.5–4× (S01: 2.28 measured against a 1.10 noise floor), leaving the mined control limits too wide ever to fire.
2. **Cycles versus wall-clock time.** Converting hours to cycles with processing time alone ignored the 4–34% of time stations spend blocked or starved, overstating every interval by 1/utilisation.

Table 12. *Recovered fraction of the true wear slope before and after filtering stopped-machine readings.*

	S01	S02	S03	S04	S05	S06
measured / true slope, all samples	0.73	0.84	0.80	0.59	0.48	0.64
excluding stopped-machine samples	**0.98**	**0.97**	**0.96**	**0.99**	**0.92**	**0.91**

The two errors compose exactly: for S06, 1/0.64 × 1/0.711 = 2.20, against an observed interval ratio of 14,757/6,667 = 2.21. Both were fixed (the -D lc.min.value filter in J3/J4, and the utilisation term in the synthesis). The failed run is reported on purpose: it was the planted ground truth that exposed it, which is the strongest argument for validating analytics against a known answer.

9 Fault Tolerance

Hadoop's design assumes machines fail. scripts/failover.ps1 demonstrates this: it starts J3, then `docker kills` worker2, taking its DataNode and NodeManager down together with no clean shutdown, while the job is running. It then polls the cluster, waits for the job, compares its output with the undisturbed run, waits for HDFS to restore full replication and restarts the worker.

Table 13. *Fault-injection results.*

Event	Measured
worker2 killed	J3 at 2% map progress
NameNode declares the DataNode dead	~65 s after the kill
Map tasks re-run elsewhere by YARN	3 (45 attempts launched for 42 splits)
J3 completes	172 s after the kill; output **byte-identical** to the undisturbed run
Under-replicated blocks	4, back to 0 within ~10 s; every block again has 2 replicas

Two mechanisms are visible. YARN's ApplicationMaster notices the lost NodeManager's containers and re-schedules their map tasks on the surviving workers, reading the second replica of each input block. Independently, the NameNode stops receiving heartbeats, marks the DataNode dead, finds the blocks that now have one replica and copies them to another node. Detection was tuned to about 60 s so the demonstration fits in a viva; Hadoop's 10.5-minute default exists to ride out garbage-collection pauses and network blips on a production cluster.

10 Dashboard

An interactive Streamlit dashboard (dashboard/app.py) presents the results. It reads the committed job outputs in artifacts/, so six of its seven tabs work without the cluster; only the Cluster tab needs it, and shows a notice when it is unreachable. It is started with `python -m streamlit run dashboard/app.py`.

Its tabs are: **Overview** (headline metrics); **Line analytics** (busy / blocked / starved time from J2 and the OEE breakdown from J1); **Wear & SPC** (degradation curves from J4, mined vs baseline limits from J3); **Policy** (every setpoint Hadoop changed); **Closed loop** (the A/B results, plus what-if controls that re-run the synthesis with a different maintenance target or σ and score the new policy on the simulator live); **Validation** (every mined value against the ground truth); and **Cluster** (live HDFS and YARN state from the NameNode and ResourceManager REST APIs, kept open during the failover demo). The what-if controls re-synthesise from the saved job outputs; new mined results from new data still require the cluster.

11 Discussion, Limitations and Future Work

11.1 Is this big data?

The demonstration data set (145 MB) is deliberately laptop-sized so the pipeline runs in minutes; it already exceeds what a spreadsheet can open but would fit on one machine. The architecture is what scales: the same jobs run unchanged on the simulator's larger setting (30 days at 5 Hz, about 104 M rows), and a real plant of a few hundred robots sampling dozens of signals at high rates would produce on the order of a terabyte a day, which is the regime HDFS and MapReduce are designed for.

11.2 Limitations

- **The plant is simulated.** The functional forms follow standard models (linear wear, lognormal cycle times, Gaussian noise, Weibull-style wear-out, exponential repairs, the tandem-line recursion), but the coefficients are synthetic.
- **Statistical power.** Twenty seeds give a wide interval on the downtime effect (±12%); the direction is stable across runs, but a tighter estimate would need about 60 seeds.
- **No streaming tier.** Level 2 reacts in real time only through static limits. A streaming layer beside the batch tier is the honest architecture for sub-second decisions.
- **Policies are mined, not optimised.** The synthesis applies fixed rules to aggregates; it does not search for the policy that maximises an objective.
- **Only torque drives decisions.** The other three sensors are generated and filtered but not yet used.

11.3 Future work

- A streaming tier (e.g. Spark Structured Streaming or Flink) for Level 2 alerts, sharing HDFS with the batch tier.
- A Level 3 optimiser that searches maintenance targets and buffer sizes against an explicit cost model instead of fixed rules.
- Multi-sensor condition monitoring: J3 and J4 already accept any signal through -D lc.signal, so temperature and vibration can be mined without code changes.
- Validation on real plant data once telemetry with a controllable policy is available.

12 Conclusion

The project shows that Apache Hadoop can serve as the supervisory tier of a factory control system when it is placed where batch analytics belongs: at ISA-95 Level 3, setting the parameters that faster controllers enforce. Four MapReduce jobs, each built around a distinct technique (combiner, secondary sort, stable parallel variance with mapper-side filtering, and map-side join), turn 4.84 million telemetry rows into a handful of aggregates. A short synthesis step turns those into a new policy, and the loop closes when the line runs under it.

The results are validated three ways: against ground truth planted in the plant (the right bottleneck, the right fastest-wearing station, exact OEE, intervals within 2–10%), by a paired closed-loop test (throughput +1.01%, downtime -18.1%, a small OEE trade-off), and by surviving the loss of a worker mid-job with identical output. The most instructive result was a failure: the first mined policy was worse than the baseline, and only the ground-truth checks revealed why.

References

[1] ANSI/ISA-95.00.01 and IEC 62264-1:2013, *Enterprise-control system integration — Part 1: Models and terminology*.

[2] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," *Proc. 6th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2004.

[3] S. Ghemawat, H. Gobioff and S.-T. Leung, "The Google file system," *Proc. 19th ACM Symposium on Operating Systems Principles (SOSP)*, 2003.

[4] K. Shvachko, H. Kuang, S. Radia and R. Chansler, "The Hadoop Distributed File System," *Proc. IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST)*, 2010.

[5] V. K. Vavilapalli et al., "Apache Hadoop YARN: Yet another resource negotiator," *Proc. 4th ACM Symposium on Cloud Computing (SoCC)*, 2013.

[6] T. White, *Hadoop: The Definitive Guide*, 4th ed., O'Reilly Media, 2015.

[7] S. Nakajima, *Introduction to TPM: Total Productive Maintenance*, Productivity Press, 1988.

[8] E. M. Goldratt and J. Cox, *The Goal: A Process of Ongoing Improvement*, North River Press, 1984.

[9] J. A. Buzacott and J. G. Shanthikumar, *Stochastic Models of Manufacturing Systems*, Prentice Hall, 1993.

[10] D. C. Montgomery, *Introduction to Statistical Quality Control*, 7th ed., Wiley, 2013.

[11] T. F. Chan, G. H. Golub and R. J. LeVeque, "Algorithms for computing the sample variance: analysis and recommendations," *The American Statistician*, vol. 37, no. 3, 1983.

[12] A. Bahga and V. K. Madisetti, "Analyzing massive machine maintenance data in a computing cloud," *IEEE Transactions on Parallel and Distributed Systems*, vol. 23, no. 10, 2012.

[13] Y. Bao et al., "Massive sensor data management framework in cloud manufacturing based on Hadoop," *Proc. IEEE International Conference on Industrial Informatics (INDIN)*, 2012.

[14] M.-K. Zheng et al., "MapReduce based parallel Bayesian network for manufacturing quality control," *Chinese Journal of Mechanical Engineering*, vol. 30, 2017.

[15] J. Zhu et al., "Monitoring big process data of industrial plants with multiple operating modes based on Hadoop," *Journal of the Taiwan Institute of Chemical Engineers*, 2018.

[16] S. Patwardhan, A. K. Verma and U. Kumar, "A survey on predictive maintenance through big data," in *Current Trends in Reliability, Availability, Maintainability and Safety*, Springer, 2016.

[17] The Apache Software Foundation, *Apache Hadoop 3.4.1 documentation*, hadoop.apache.org.